Scope 3 Emissions Estimation from Procurement Text: Project Plan

Rishik Kolli (rk5129), Simranjeet Singh (xs2797), Bineet Singh Chadha (bsc6177)

Refined Problem and Task

Each input is a single purchase record: a free-text description, the merchant or recipient, an optional category field (the merchant category code on a purchase card, or the product code on a federal contract), a dollar amount and a date. For each record, the system assigns a 2022 NAICS-6 code and computes emissions by multiplying the spend, converted to 2022 dollars, by the matching EPA v1.3 emission factor. It also reports how confident it is. When confidence is low, the system abstains and sends the item to a human reviewer, and the review queue is ordered by how much emissions error each item could cause. Two things have changed since the proposal. Abstention is now a core part of the project rather than an extension, and the evaluation set now mixes clean federal contracts with messy purchase-card data that we label ourselves. We plan to spend most of our effort on retrieval quality, calibration and error analysis rather than on prompt wording.

Response to Proposal Feedback

Can one definition map to several codes? We checked the Census files directly. All 1,012 six-digit definitions are distinct, and none of the 20,398 index phrases maps to more than one code, so the definitions themselves are not ambiguous. The ambiguity appears on the query side instead: 1,076 head nouns in the index span several codes (for example, *pipe*, *bags* and *furniture* each appear under ten), and a two-word purchase line like "pipe" inherits that problem. We will therefore index each phrase as its own chunk, combine scores by code, and show the model the Census cross-references ("except X, see code Y") as hints about what a code is not. The mapping also runs the other way: 1,016 codes share only 281 distinct EPA factors, so some wrong codes cost nothing in emissions. We will report this as a separate factor-equivalent accuracy.

How low will similarity scores be? We expect cosine scores between short, messy text and long formal definitions to be low and tightly bunched. A typical purchase-card line has two words, while a definition has 47. For that reason we will never use a raw similarity score as a threshold or as a confidence signal. Our first experiment measures the actual score distributions for correct and incorrect codes, and we will judge retrieval by whether the right code appears in the top k results (recall@k). To close the gap in length and style, we will test short index-phrase chunks, keyword (BM25) search alongside embeddings, LLM rewriting of the query, and a reranker.

How will the dollar amount be used? The amount has four roles. First, it drives the emissions estimate itself. Second, it sets review priority: an item's priority is its spend times the chance that the code is wrong times the spread of emission factors among the candidate codes, which matters because factors range from 0.03 to 3.92 kgCO2e per dollar. Third, we will test it as a model input, since a $40 software line is probably a license while a $4M one is probably custom development. Fourth, it needs careful handling: the purchase-card data includes 1,281 refunds, and single federal awards reach $12.6B, which would dominate any summed error.

Is the evaluation set too clean? We agree that federal contracts are cleaner than corporate expense data. The test set will contain 300 federal contracts and 200 hand-labeled purchase-card transactions, and every result will be reported separately for each. We will also set aside 100 labeled purchase-card rows for calibration, so the abstention threshold is not tuned on clean data alone.

Simplest Credible Baselines

We will compare against four baselines. The simplest takes the top result from search over the NAICS text, with no language model at all. The second is a TF-IDF and logistic regression classifier trained on about 6,400 labeled federal contracts, which tells us whether generative AI is needed in the first place. The third is the zero-shot LLM from our proposal. For purchase-card data only, the fourth is a hand-built mapping from the most common merchant categories to NAICS codes.

Proposed Architecture

The system first cleans each description by removing contract boilerplate and, optionally, asking an LLM to rewrite it in plain language. It then searches a corpus of NAICS titles, definitions, index phrases and cross-references using both embeddings (e5 or bge) and keyword search, merges the two rankings, groups results by code and reranks the top 50. The merchant or product category can give a small boost to related codes, following the metadata-filtered RAG pattern from week 7. An LLM then picks one code from the top candidates, guided by similar labeled examples, and returns its choice, a short rationale and a confidence score as JSON. We estimate confidence from four signals: agreement across five sampled answers, the probability of the chosen code, the score gap between the top two candidates, and the model's stated confidence. A logistic regression fit on held-out data combines them. Finally, the code is mapped to the 2017 codes that the EPA factors use. Every stage logs its inputs and outputs, so we can trace each error to the step that caused it.

Evaluation Dataset and Test Cases

Split	Source	Size	Labels and purpose
gov_pool	USAspending FY2024 contracts (6,900 pulled across 23 sectors)	~6,400	Award NAICS codes; used for few-shot examples, the supervised baseline and calibration
gov_test	USAspending, stratified by sector and held out	300	Award NAICS codes; we re-label 50 blind to check them
messy_dev, messy_test	Oklahoma purchase cards, July 2024, stratified by category	100, 200	Two of us label each row and the third settles disagreements; we report Cohen's κ
challenge	Hand-picked from both sources	~40	The edge cases below, re-run after every change

Success Metrics and Evaluation Methodology

Our main metric is emissions error: the total absolute difference between predicted and true kgCO2e, divided by the true total. Because a few very large awards could dominate that number, we will also report the per-item median. Secondary metrics are accuracy at the 2-, 4- and 6-digit levels, factor-equivalent accuracy, recall@k for retrieval, and cost and latency per item. For abstention, we will measure how well confidence separates right from wrong answers (AUROC), the error rate at each level of coverage (the risk-coverage curve and its area), calibration error (ECE), and emissions error when the system keeps 70, 80 or 90 percent of items. To show that confidence tracks what is actually wrong, we will check on the hand-labeled data whether low model confidence lines up with items where our annotators disagreed, and whether the system abstains on items that we marked as unknowable. All comparisons will include 95% bootstrap confidence intervals. We will consider the project successful if it beats the supervised and zero-shot baselines on emissions error for both data types and reaches an AUROC of at least 0.75 on the purchase-card test set.

Outcome vs. Trajectory Evaluation

Outcome evaluation asks whether the final code, emissions estimate and abstain decision are right. Trajectory evaluation asks where things went wrong along the way, following the logging approach from the week 8 agent assignment. For each item we will check whether the correct code was retrieved and at what rank, whether query rewriting helped or hurt, whether the LLM picked the correct code when it was available, and whether the confidence score flagged the mistake. This lets us assign every error to retrieval, selection, label noise or a missed abstention, and report each category by count and by its share of emissions error. If we build the agentic stretch version, we will also look at how many tool calls it makes and how many of them are wasted.

Important Edge Cases

The biggest challenge in the purchase-card data is that 39% of rows have no useful description, such as "GENERAL PURCHASE" or "PAYMENT ON ACCOUNT". For these, the system has to rely on the merchant and category and should usually abstain. Another 18% of rows are Amazon purchases, where the merchant category says little about what was bought, and purchases from retailers like Walmart should be coded to the industry that made the product, not the store. Some lines combine several items or are cut off mid-word ("Hea PCE"). Federal descriptions bring their own problems, including contract-number boilerplate, a few nearly empty descriptions, and labels that mix 2017 and 2022 NAICS codes. Across both sources we also need to handle refunds, very large awards, and items like software or HVAC where the line between a good and a service is unclear.

Planned Experiments and Comparisons

We plan six experiments. The first measures similarity score distributions. The second adds retrieval improvements one at a time and tracks recall@k: query prefixes, index-phrase chunks, keyword search, query rewriting, reranking and the category boost. The third compares the baselines with retrieval-augmented classification, with and without few-shot examples, the dollar amount and the category field, and at different numbers of candidates. The fourth compares confidence signals on their own and combined, and calibrated on clean versus messy data. The fifth compares ordering the review queue by probability of error against ordering it by expected emissions error. As a stretch, the sixth compares small and large models.

Core and Stretch Goals

Our core goals are the NAICS corpus and overlap analysis, the hybrid retriever and its ablations, the baselines and the retrieval-augmented classifier, confidence-based abstention with full calibration metrics, 300 double-labeled purchase-card rows, and an error analysis broken down by stage and weighted by emissions. Our stretch goals are an embedding model fine-tuned on federal contract data, a small LoRA-tuned classifier (building on week 9), hierarchical classification from 2 to 4 to 6 digits, an agent that can re-run searches, and a simple demo of the review queue.

Team Ownership and Division of Work

Rishik will own the NAICS corpus, the overlap analysis, the retrieval experiments and the data pipeline. Simranjeet will own the LLM classifier, few-shot examples, query rewriting, the emissions calculation and the baselines. Bineet will own the confidence signals, abstention, and the evaluation harness and statistics. Labeling is shared: each of us will label about 200 purchase-card rows so that every row gets two independent labels, Rishik will settle disagreements, and Simranjeet and Bineet will split the 50-row federal audit. Each of us will write the error analysis for the part of the system we own.

Next Steps

In the first week, we will freeze the data splits, draw the purchase-card sample, label 30 rows together to agree on the guidelines, and build the corpus. In the second week, we will run the retrieval experiments and the first baselines and finish the first round of labeling. The third week covers the retrieval-augmented classifier, the rest of the labeling and the federal audit. The fourth week is for confidence and abstention, and the fifth is for error analysis, stretch goals and the final report.